

An Interactive Approach to Extrinsic Calibration of 3D LiDAR and Monocular Camera Using Open Source Toolchain

Comprehensive Tutorial Based on Paracha et al. (2025)

Abstract

In robotics, autonomous driving, and intelligent systems, sensor fusion between 3D LiDAR and monocular cameras is essential for robust environmental perception. This tutorial presents a detailed, step-by-step guide to the **interactive extrinsic calibration method** proposed by Paracha et al., which leverages an **ArUco marker**, open-source tools (OpenCV, Open3D, ROS2), and a user-friendly workflow to estimate the transformation matrix between LiDAR and camera coordinate frames.

The method achieves **reprojection errors below one pixel** and works reliably across high- and low-resolution sensors. It is designed for practical deployment in real-world scenarios and is fully open-source, with code and dataset available at:

<https://github.com/tui-abdul/3d-lidar-camera-calib>

This document serves as a complete reference for implementing the calibration pipeline, suitable for generating a printable PDF guide.

Table of Contents

1. Introduction
2. Motivation and Key Features
3. Prerequisites and Hardware Setup
4. Software Environment and Dependencies
5. Step-by-Step Calibration Procedure
 - 5.1 Intrinsic Camera Calibration
 - 5.2 Data Acquisition
 - 5.3 Feature Extraction – Camera
 - 5.4 Feature Extraction – LiDAR
 - ROI Selection

- Plane Fitting (RANSAC)
- Layer-wise Line Fitting
- Edge Filtering and Corner Detection

5.5 Optimization: SolvePnP Ransac + EPnP

5.6 Visualization and Validation

6. Evaluation and Results

6.1 Experimental Design

6.2 Translation Comparison

6.3 Reprojection Error Analysis

6.4 Visual Inspection

7. Best Practices and Tips

8. Limitations and Future Work

9. Conclusion

10. References

1. Introduction

Sensor fusion of **3D LiDAR** and **monocular camera** data enables accurate 3D object detection, depth estimation, and scene understanding in autonomous systems. However, this fusion requires precise **extrinsic calibration** – the determination of the rotation R and translation t that transform points from the LiDAR frame to the camera frame.

This calibration defines the **extrinsic matrix**:

$$M = [R \mid t]$$

Without accurate extrinsics, projected LiDAR points will misalign with camera images, degrading perception performance.

This tutorial explains the **target-based**, **interactive**, and **open-source** approach introduced by Paracha et al. (2025), which uses an **ArUco marker** to establish 3D–2D correspondences and computes the extrinsic parameters using **EPnP + RANSAC** optimization.

2. Motivation and Key Features

Why This Method?

While deep learning-based methods exist (e.g., RegNet, CalibNet), they require large datasets and training. Target-less methods lack robustness under varying conditions.

This work offers a **practical, accessible alternative** with the following advantages:

- **Open-source toolchain:** Built with Python, ROS2, OpenCV, Open3D
- **Interactive interface:** Handles noisy or sparse LiDAR data
- **Works with any resolution:** High- or low-end LiDARs and cameras
- **No need for intensity or color:** Uses only geometry
- **Accurate results:** Reprojection error < 1 pixel
- **Publicly available:** Code and dataset shared

Core Idea

Use an **ArUco marker** as a calibration target:

- Detect **2D corners and center** in camera image automatically
- Extract corresponding **3D points** from LiDAR via interactive processing
- Compute extrinsics using **EPnP with RANSAC**
- Validate via reprojection and manual translation check

3. Prerequisites and Hardware Setup

Hardware Requirements

Component	Example	Notes
3D LiDAR	Ouster OS-1	Must provide XYZ + ring/channel info
Monocular Camera	Valeo CAM 1MP TO KIT	Global shutter preferred
Mounting Platform	Aluminum profile	Rigid, stable mounting
Calibration Target	ArUco marker (800×800 mm)	White background, black border

Tip: Larger markers improve LiDAR segmentation accuracy.

Coordinate Systems

- **LiDAR (ISO 8855):**
 - X: Forward (+)
 - Y: Left (+)
 - Z: Upward (+)

- **Camera (Pinhole Model):**

- X: Right(+)
- Y: Down(+)
- Z: Forward(+)

Ensure both sensors are fixed and non-moving during calibration.

4. Software Environment and Dependencies

Operating System

- Ubuntu 22.04 LTS
- ROS 2 Humble Hawksbill

Required Libraries

```
pip install opencv-python open3d numpy scipy matplotlib
```

ROS2 Packages

- `ros2 run camera_calibration cameracalibrator.py` – for intrinsic calibration
- `message_filters.ApproximateTimeSynchronizer` – for sensor synchronization
- `cv2.aruco.ArucoDetector` – for ArUco detection
- `open3d.visualization.VisualizerWithVertexSelection` – for point cloud interaction

Code Repository

<https://github.com/tui-abdul/3d-lidar-camera-calib>

Clone the repository:

```
git clone https://github.com/tui-abdul/3d-lidar-camera-calib.git
cd 3d-lidar-camera-calib
```

5. Step-by-Step Calibration Procedure

5.1 Intrinsic Camera Calibration

Before extrinsic calibration, perform **intrinsic calibration** to obtain:

- Intrinsic matrix K
- Distortion coefficients D

Using ROS2 Camera Calibration Tool

Launch the calibration node:

```
ros2 run camera_calibration cameracalibrator.py \
  --size 8x6 --square 0.024 \
  image:=/camera/image_raw camera:=/camera
```

Save the result as `ost.yaml`. Extract:

```
K = [[fx, 0, cx],
      [0, fy, cy],
      [0, 0, 1]]
```

```
D = [k1, k2, p1, p2, k3]
```

Alternative: Use OpenCV's `calibrateCamera()` or MATLAB.

5.2 Data Acquisition

Collect synchronized LiDAR point clouds and camera images with the **ArUco marker** visible in both.

Synchronization (ROS2)

Use `ApproximateTimeSynchronizer`:

```
from message_filters import ApproximateTimeSynchronizer, Subscriber

lidar_sub = Subscriber('/lidar/points', PointCloud2)
image_sub = Subscriber('/camera/image_raw', Image)

ts = ApproximateTimeSynchronizer([lidar_sub, image_sub], queue_size=10, s
ts.registerCallback(callback)
```

Data Collection Strategy

- Place marker at **6 different positions**, 2–6 meters away
- Record **30 seconds per position** (ROS2 bag file recommended)
- Capture **5 frames per position** for testing
- Total: ~150 feature pairs

Note: One frame per position may suffice if marker is large and well-aligned.

5.3 Feature Extraction – Camera

Automatically detect **2D pixel coordinates** of the ArUco marker.

Using OpenCV ArUco Detector

```
import cv2
import cv2.aruco as aruco

def detect_aruco_features(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    dictionary = aruco.getPredefinedDictionary(aruco.DICT_4X4_50)
    parameters = aruco.DetectorParameters()
    detector = aruco.ArucoDetector(dictionary, parameters)

    corners, ids, rejected = detector.detectMarkers(gray)

    if len(corners) == 0:
        return None # No marker found

    corner_pts = corners[0][0] # Shape: (4, 2)
    center_pt = np.mean(corner_pts, axis=0)

    return corner_pts, center_pt
```

Output: 5 points – 4 corners + 1 center in **image coordinates (u, v)**

5.4 Feature Extraction – LiDAR

Due to LiDAR sparsity and noise, this step is **interactive**.

Step 1: ROI Selection (Interactive)

Load point cloud and select region containing the marker:

```
import open3d as o3d

pcd = o3d.io.read_point_cloud("frame.ply")
vis = o3d.visualization.VisualizerWithVertexSelection()
vis.create_window()
vis.add_geometry(pcd)
vis.run()
selected_indices = [point.index for point in vis.get_picked_points()]
vis.destroy_window()

selected_pcd = pcd.select_by_index(selected_indices)
```

Step 2: Plane Fitting (RANSAC)

Fit a plane to reduce multi-surface noise:

```
plane_model, inliers = selected_pcd.segment_plane(
    distance_threshold=0.01,
    ransac_n=3,
    num_iterations=1000
)
[a, b, c, d] = plane_model

# Project all points onto the plane
points = np.asarray(selected_pcd.points)
distances = (a*points[:,0] + b*points[:,1] + c*points[:,2] + d) / np.sqrt
projected_points = points - distances[:, None] * np.array([a,b,c])
```

Step 3: Layer-wise Line Fitting

LiDAR organizes points into **rings/channels**. Fit a line per ring:

```
unique_rings = np.unique(ring_ids)
for ring_id in unique_rings:
    ring_mask = (ring_ids == ring_id)
    ring_points = points[ring_mask]

    # Fit 3D line using SVD
    centroid = np.mean(ring_points, axis=0)
    centered = ring_points - centroid
```

```
_, _, vh = np.linalg.svd(centered)
direction = vh[0]

# Project points onto line
t = np.dot(centered, direction)
projected = centroid + np.outer(t, direction)
```

Step 4: Edge Filtering and Corner Detection

- Filter **boundary points** using ring discontinuities
- **Interactively select** each edge using Open3D
- Fit a **3D line** to each selected edge
- Compute **corner points** as **intersection of two adjacent edge lines**

```
def line_intersection_3d(line1, line2):
    # Implementation using least squares or geometric solver
    ...
```

- Compute **center point** as mean of all marker points

Output: 5 feature points in **LiDAR coordinates (x, y, z)**

5.5 Optimization: SolvePnP Ransac + EPnP

Now solve for extrinsic parameters using 3D-2D correspondences.

Input to `solvePnP Ransac`

- `objectPoints` : 3D LiDAR points (Nx3)
- `imagePoints` : 2D camera points (Nx2)
- `cameraMatrix` : Intrinsic K
- `distCoeffs` : Distortion D
- `flags` : `cv2.SOLVEPNP_EPnP`
- `reprojectionError` : 1.0 pixel
- `confidence` : 0.99
- `iterationsCount` : 1,000,000

```
ret, rvec, tvec, inliers = cv2.solvePnP Ransac(
    objectPoints=points_3d,
    imagePoints=points_2d,
```



```

cameraMatrix=K,
distCoeffs=D,
reprojectionError=1.0,
confidence=0.99,
flags=cv2.SOLVEPNP_EPNP
)

```

Refinement with Levenberg-Marquardt

```

ret, rvec, tvec = cv2.solvePnPRefineLM(
    objectPoints=points_3d[inliers],
    imagePoints=points_2d[inliers],
    cameraMatrix=K,
    distCoeffs=D,
    rvec=rvec,
    tvec=tvec
)

```

Final output: **rvec** (rotation vector), **tvec** (translation vector)

5.6 Visualization and Validation

Reproject LiDAR Points to Image

```

reprojected, _ = cv2.projectPoints(
    points_3d, rvec, tvec, K, D
)

for pt in reprojected:
    u, v = int(pt[0][0]), int(pt[0][1])
    cv2.circle(image, (u, v), 3, (0, 255, 0), -1)

```

Overlay on original image to verify alignment with ArUco edges.

Validation Criteria

1. Reprojection Error (RMSE)

\$\$

$$e = \sqrt{\frac{1}{N} \sum_{i=1}^N |x_i - P(X_i)|^2}$$

\$\$

Should be < 1 pixel

2. Manual Translation Check

Measure physical distance between LiDAR and camera (e.g., 12.4 cm)

Compare with $\$|t_{vec}|\$$: should be close

3. Visual Inspection

Reprojected points should align with object edges in image

6. Evaluation and Results

6.1 Experimental Design

Parameter	Value
LiDAR	Ouster OS-1
Camera	Valeo CAM 1MP TO KIT
Marker Size	800×800 mm
Positions	6
Frames per Position	5
Total Points	150
Optimization	EPnP + RANSAC
Reprojection Threshold	1.0 px
Confidence	0.99
Iterations	1,000,000

6.2 Translation Comparison

Case	Manual (cm)	Estimated (cm)	Difference
150 points	12.406	14.00	+1.594
30 points	12.406	13.31	+0.904

Despite small overestimation, results are considered acceptable due to visual alignment.

6.3 Reprojection Error (RMSE)

Case	RMSE (pixels)
150 points	0.936
30 points	0.8126

Both cases achieve **sub-pixel accuracy**

6.4 Visual Inspection

As shown in Fig. 7 of the paper, reprojected LiDAR points align well with object edges in the image. Intensity values help distinguish surfaces.

Tip: Use intensity coloring to verify edge alignment.

7. Best Practices and Tips

Tip	Description
Use large ArUco marker	≥800 mm improves LiDAR segmentation
White background	Required for reliable ArUco detection
Avoid occlusion	Marker must be fully visible
Multiple positions	At least 6 positions recommended
Single frame per position	Often sufficient with large marker
Check lighting	Avoid glare or shadows on marker
Save intermediate data	Store feature points for debugging

8. Limitations and Future Work

Current Limitations

- **Manual interaction** increases calibration time
- Requires **white background** for ArUco detection
- Not fully automated

- Sensitive to marker visibility

Future Improvements

- Automate ROI and edge selection using deep learning
 - Integrate **Segment Anything Model (SAM)** for zero-training detection
 - Support dynamic environments
 - Benchmark against state-of-the-art (RegNet, CalibNet)
-

9. Conclusion

This tutorial provides a complete implementation guide for the **interactive extrinsic calibration method** between 3D LiDAR and monocular camera using open-source tools. The approach is:

- **Robust:** Handles noisy and sparse LiDAR data
- **Accurate:** Achieves sub-pixel reprojection error
- **Flexible:** Works across sensor resolutions
- **Open-source:** Publicly available code and dataset

By following this guide, researchers and engineers can reliably calibrate their sensor setups for applications in robotics, autonomous driving, and 3D perception.

10. References

- [1] S. Thrun et al., "Stanley: The robot that won the DARPA Grand Challenge," *Journal of Field Robotics*, vol. 23, no. 9, pp. 661–692, 2006.
 - [2] V. Lepetit et al., "EPnP: An Accurate $O(n)$ Solution to the PnP Problem," *Int J Comput Vis*, vol. 81, no. 2, pp. 155–166, 2009.
 - [3] A. Elaksher et al., "A Quantitative Assessment of LiDAR Data Accuracy," *Remote Sensing*, vol. 15, no. 2, 2023.
 - [4] N. Schneider et al., "RegNet: Multi-modal sensor registration using deep neural networks," *IV 2017*.
 - [5] G. Iyer et al., "CalibNet: Geometrically Supervised Extrinsic Calibration," *IROS 2018*.
 - [6] Z. Luo et al., "Calib-Anything: Zero-training LiDAR-Camera Extrinsic Calibration," arXiv:2306.02656, 2023.
 - [7] M. A. Fischler and R. C. Bolles, "Random sample consensus," *Commun. ACM*, vol. 24, no. 6, 1981.
 - [8] L. Zhou et al., "Automatic extrinsic calibration of a camera and a 3D LiDAR," *IROS 2018*.
 - [9] M. Kalaitzakis et al., "Fiducial Markers for Pose Estimation," *J Intell Robot Syst*, vol. 101, no. 4, 2021.
 - [10] D. W. Marquardt, "An Algorithm for Least-Squares Estimation of Nonlinear Parameters," *J. SIAM*, vol. 11, no. 2, 1963.
-

Authors:

Abdul Haq Azeem Paracha, Christoph Brückner, Prof. Dr. Lucila Patiño Studencki, Prof. Dr.-Ing. Georg Arbeiter

Mechanical and Automotive Engineering Department
Hochschule Coburg – University of Applied Sciences
Kronach, Germany

Affiliation: Part of the DEKOR-X project funded by BMWK (German Federal Ministry for Economic Affairs and Climate Action)

Code & Dataset:

<https://github.com/tui-abdul/3d-lidar-camera-calib>

© 2025 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses.

DOI: 10.1109/SIMPARR62925.2025.10978990